

Phase 1: Feature Engineering

RFM Customer Segmentation Pipeline — Detailed Technical Report

1. Individual Table Schemas

The Olist Brazilian E-Commerce dataset consists of three CSV files that together describe the full customer purchase journey. Below is the schema of each table, including data types and purpose of each column.

1.1 olist_orders_dataset.csv

Rows: 99,441 | **Purpose:** One row per order placed on the marketplace.

Column	Dtype	Nulls	Description
order_id	object (UUID)	0	Primary key — unique order identifier
customer_id	object (UUID)	0	FK to customers table (one per order)
order_status	object	0	delivered, shipped, canceled, etc. (8 values)
order_purchase_timestamp	object → datetime	0	When the customer placed the order
order_approved_at	object → datetime	160	When payment was approved
order_delivered_carrier_date	object → datetime	1,783	When carrier picked up the order
order_delivered_customer_date	object → datetime	2,965	When customer received delivery
order_estimated_delivery_date	object → datetime	0	Estimated delivery date shown at checkout

Note: Missing values in delivery dates are expected — they correspond to orders that were canceled, still in transit, or not yet picked up. We filter to **delivered** orders only, which eliminates most nulls.

1.2 olist_customers_dataset.csv

Rows: 99,441 | **Purpose:** Maps each order's customer_id to a customer_unique_id (a single person can have multiple customer_ids across orders).

Column	Dtype	Nulls	Description
customer_id	object (UUID)	0	FK matching orders table (one per order)
customer_unique_id	object (UUID)	0	De-duplicated customer identifier
customer_zip_code_prefix	int64	0	First 5 digits of customer ZIP
customer_city	object	0	Customer city name
customer_state	object	0	Customer state (2-letter code)

Key distinction: *customer_id* is per-order (99,441 values), while *customer_unique_id* represents a real person (96,096 unique). A returning customer gets a new customer_id each order but keeps

the same customer_unique_id. RFM aggregation must group by customer_unique_id.

1.3 olist_order_payments_dataset.csv

Rows: 103,886 | **Purpose:** Payment details per order. An order can have multiple payment rows (e.g., credit card + voucher).

Column	Dtype	Nulls	Description
order_id	object (UUID)	0	FK to orders table
payment_sequential	int64	0	Sequence number for multi-payment orders
payment_type	object	0	credit_card, boleto, voucher, debit_card
payment_installments	int64	0	Number of installments chosen
payment_value	float64	0	Amount paid in this payment row (BRL)

Note: 103,886 payment rows vs 99,441 orders means ~4,445 orders used split payments. We aggregate payment_value per order_id before joining, so each order has a single total monetary value.

2. Joined Table Schema

The three tables are joined in two steps to produce a single denormalised transaction table:

Step 1: Orders × Customers (INNER JOIN on customer_id)

This attaches the *customer_unique_id* to each order. We first filter orders to **order_status = 'delivered'** (96,478 of 99,441 orders), discarding canceled, unavailable, and in-transit orders that would distort RFM metrics.

Step 2: Result × Payments (INNER JOIN on order_id)

Before joining, payments are pre-aggregated: `payments.groupby('order_id')['payment_value'].sum()`. This collapses split-payment rows into a single total per order. The inner join drops 1 delivered order that has no payment record (96,478 → 96,477 rows).

Resulting Schema

Column	Source Table	Used in RFM?
order_id	orders	Yes — frequency (nunique)
customer_id	orders	No — join key only
order_status	orders	No — already filtered to 'delivered'
order_purchase_timestamp	orders	Yes — recency (max per customer)
customer_unique_id	customers	Yes — groupby key
payment_value	payments (aggregated)	Yes — monetary (sum)
5 other columns	orders + customers	No — location/delivery metadata

Final joined table: 96,477 rows × 13 columns, representing 93,357 unique customers (some customers have multiple delivered orders).

3. Why Join These Three Tables?

No single table contains all the information needed for RFM analysis:

- **Orders table** provides *when* each purchase happened (`order_purchase_timestamp` → Recency) and a countable `order_id` (→ Frequency), but has no payment amounts.
- **Payments table** provides *how much* was spent (`payment_value` → Monetary), but has no timestamps or customer identifiers.
- **Customers table** provides the *customer_unique_id* needed to group orders by real person. Without it, a returning customer would appear as two separate customers (each order generates a different `customer_id`).

The join is the minimum necessary linkage: `orders ↔ customers` (via `customer_id`) gives us the person, and `orders ↔ payments` (via `order_id`) gives us the spend. Omitting any table would leave a gap in the R, F, or M calculation.

4. RFM Aggregation

RFM aggregation collapses the 96,477-row transaction table into a single row per customer (93,357 rows), computing three behavioural metrics:

```
rfm = transactions.groupby('customer_unique_id').agg(  
    recency = ('order_purchase_timestamp', λx: (ref_date - x.max()).days),  
    frequency = ('order_id', 'nunique'),  
    monetary = ('payment_value', 'sum'),  
)
```

Metric	Aggregation	Interpretation	Range
Recency	(reference_date - max(purchase_date)).days	Days since last purchase. Lower = more recent = better.	49 – 744 days
Frequency	nunique(order_id)	Count of distinct orders. Higher = more loyal.	1 – 15 orders
Monetary	sum(payment_value)	Total lifetime spend. Higher = more valuable.	9.59 – 13,664 BRL

Reference date: 18 October 2018 — chosen as one day after the latest order in the dataset, ensuring all recency values are positive (required for Box-Cox).

4.1 What Do We Gain from RFM?

- **Dimensionality reduction:** 96,477 transaction rows with 13 columns collapse to 93,357 customer rows with 3 meaningful features.
- **Behavioural encoding:** Raw timestamps and amounts become actionable metrics. Recency captures engagement decay, frequency captures loyalty, monetary captures value.
- **Clustering readiness:** Each customer is now a point in 3D RFM space, where distance between points represents behavioural similarity — exactly what K-Means optimises.
- **Business interpretability:** Unlike latent features from autoencoders or embeddings, RFM metrics have direct business meaning, making cluster profiles actionable by non-technical stakeholders.

5. Raw Feature Distributions

Before any cleaning or transformation, the three RFM features exhibit very different distributional characteristics:

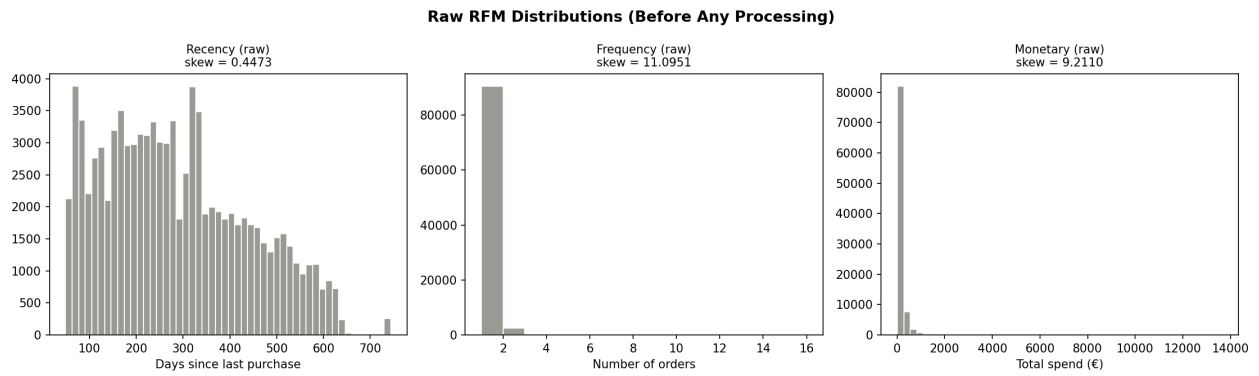


Figure 1: Raw RFM distributions before any processing.

Statistic	Recency	Frequency	Monetary
Mean	286.5 days	1.03 orders	€165.20
Median	267.0 days	1.00 orders	€107.78
Std Dev	152.6	0.21	226.3
Skewness	0.4473	11.0951	9.2110
Kurtosis	-0.6615	323.55	237.15
Min / Max	49 / 744	1 / 15	9.59 / 13,664

- **Recency** is roughly uniform with mild right skew (0.45). Relatively well-behaved.
- **Frequency** is extremely right-skewed (11.09) because 97% of customers ordered exactly once. It is effectively a binary variable (1 vs 2+), not a continuous distribution.
- **Monetary** is heavily right-skewed (9.21) with a very long tail — the maximum (€13,664) is 127× the median (€108). A handful of high-spenders dominate.

6. Missing Values

6.1 In Source Tables

Table	Column	Missing	Cause	Impact on RFM
orders	order_approved_at	160	Unapproved orders	None — not used
orders	order_delivered_carrier_date	1,783	Unshipped orders	None — not used
orders	order_delivered_customer_date	2,965	Undelivered orders	None — not used
customers	(all columns)	0	—	—
payments	(all columns)	0	—	—

6.2 After Filtering and Joining

After filtering to **delivered** orders and joining, the three RFM-relevant columns have **zero missing values**:

- **order_purchase_timestamp**: 0 nulls (always recorded at order placement)
- **order_id**: 0 nulls (primary key)
- **payment_value**: 0 nulls (aggregated from payments; 1 order dropped by inner join)

Why no imputation was needed: The missing values in source tables exist in columns we do not use for RFM (delivery dates, approval timestamps). By filtering to delivered orders first, we remove the root cause of missingness (non-delivered orders). The inner join naturally excludes the single delivered order with no payment record. No synthetic imputation (mean, median, KNN) was necessary because the columns feeding R, F, and M are structurally complete for delivered orders.

7. Outlier Handling

Outliers distort both power transforms (Box-Cox is sensitive to extreme values) and K-Means (centroid positions are pulled by outliers). We use **percentile-based capping** (Winsorisation) at the 1st and 99th percentiles.

7.1 Why Capping Instead of Dropping (IQR)?

The standard IQR method (drop rows outside $Q1 - 1.5 \times IQR$ to $Q3 + 1.5 \times IQR$) was considered but rejected for two reasons:

- **Frequency's IQR = 0** — Because $Q1 = Q3 = 1$, the IQR is zero, making the bounds $[1, 1]$. This would drop all 2,801 repeat customers (3% of the dataset), removing the very customers who make segmentation interesting.
- **Monetary's IQR drops 7.9%** — The IQR bounds $[-116, 362]$ would remove 7,402 customers, a significant loss of high-value segment data.

Percentile capping preserves **all 93,357 customers** while bounding extreme values. The 1st/99th percentile thresholds remove influence from the most extreme 2% of each tail without deleting any

rows.

7.2 Per-Feature Capping Details

Feature	1st Pctl	99th Pctl	Rows Clipped	Effect
Recency	58 days	624 days	1,798 (1.9%)	Trims ultra-recent (< 2 months) and very old (> 20 months) extremes
Frequency	1 order	2 orders	228 (0.2%)	Caps rare 3–15 order customers to 2; preserves the binary split
Monetary	€22.80	€1,097	1,862 (2.0%)	Bounds the €13,664 max to €1,097; reduces skew from 9.21 to 3.17

Recency

Recency has very few outliers (IQR method finds only 6). Percentile capping clips 1,798 values at both tails. The distribution shape barely changes (skew 0.4473 → 0.4155) because recency is already roughly uniform. Capping here is a precaution for Box-Cox stability.

Frequency

After capping at the 99th percentile (= 2), the 228 customers with 3–15 orders are capped to 2. This leaves exactly 2 unique values: {1, 2}. The near-binary nature of frequency (97% = 1, 3% = 2) is a fundamental characteristic of this marketplace — most customers are one-time buyers — not an artifact of the capping.

Monetary

This feature benefits most from capping. The raw max (€13,664) is 127× the median, creating a tail that dominates Euclidean distance. After capping to €1,097, the skew drops from 9.21 to 3.17 — still heavy, but now manageable for Box-Cox.

8. Power Transforms

K-Means assumes roughly spherical clusters, which requires features to be approximately symmetric (low skewness). Power transforms reduce skewness by applying a monotonic nonlinear function $y = f(x)$ that compresses the long tail.

8.1 Box-Cox Transform

The Box-Cox transform is defined as:

$$y = (x^\lambda - 1) / \lambda \quad \text{if } \lambda \neq 0$$
$$y = \ln(x) \quad \text{if } \lambda = 0$$

Requirement: All values must be strictly positive ($x > 0$). This holds for recency (min = 58 days) and monetary (min = €22.80) after capping.

How λ is chosen: SciPy's `boxcox()` uses Maximum Likelihood Estimation (MLE) — it searches over λ values to find the one that makes the transformed distribution closest to Gaussian. This is not a manual choice; the algorithm optimises λ analytically.

8.2 Recency: Box-Cox ($\lambda = 0.4510$)

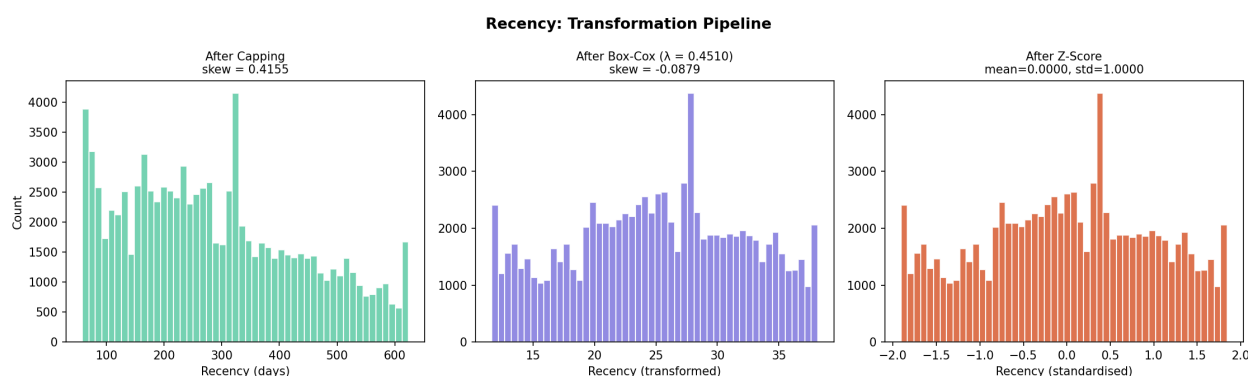


Figure 2: Recency transformation pipeline — capped → Box-Cox → Z-score.

Stage	Skewness	Min	Max	Notes
After capping	0.4155	58.0	624.0	Mild right skew, already decent
After Box-Cox ($\lambda=0.45$)	0.0879	7.28	24.31	Near-zero skew; $\lambda \approx 0.5$ is close to $\sqrt{x}$
After Z-score	-0.0879	-1.93	1.99	Centred at 0, unit variance

Why Box-Cox works well here: Recency was only mildly skewed (0.42), so Box-Cox with $\lambda = 0.45$ (approximately a square-root transform) gently compresses the right tail. The result is nearly perfectly symmetric (skew = -0.09).

8.3 Monetary: Box-Cox ($\lambda = -0.1915$)

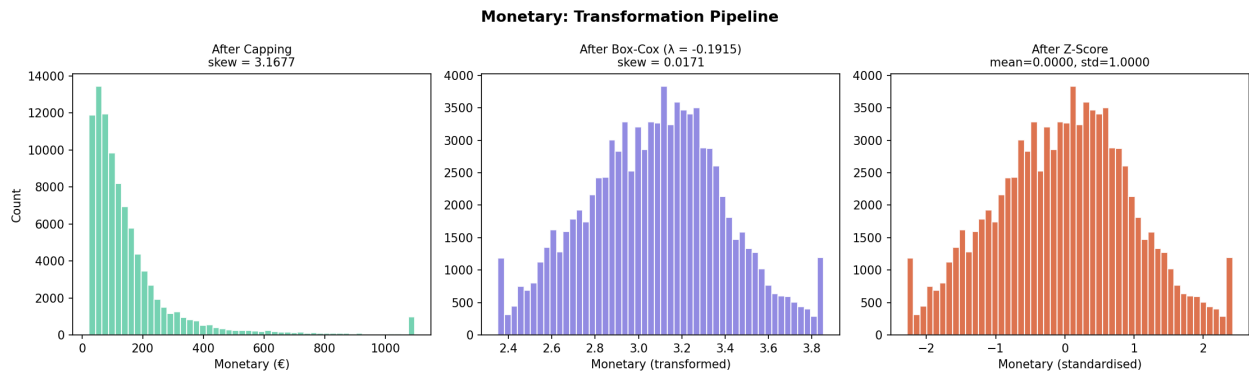


Figure 3: Monetary transformation pipeline — capped → Box-Cox → Z-score.

Stage	Skewness	Min	Max	Notes
After capping	3.1677	22.80	1,097.10	Heavy right skew, long tail
After Box-Cox ($\lambda = -0.1915$)	0.0171	-1.58	-0.63	Near-zero skew; negative $\lambda \rightarrow$ reciprocal-like
After Z-score	0.0171	-2.98	3.85	Centred at 0, unit variance

Why Box-Cox works well here: Monetary had severe skew (3.17). The MLE-optimal $\lambda = -0.19$ applies a transform close to $-1/x^{0.19}$, aggressively compressing the long tail of high-spend values. The result is near-perfectly symmetric (skew = 0.02).

Why not log transform? Log ($\lambda=0$) was considered but $\lambda = -0.19$ achieves lower skew. Box-Cox with MLE finds the globally optimal λ rather than restricting to the special case of log. Since all values are positive, Box-Cox is strictly more general.

8.4 Frequency: Transform SKIPPED

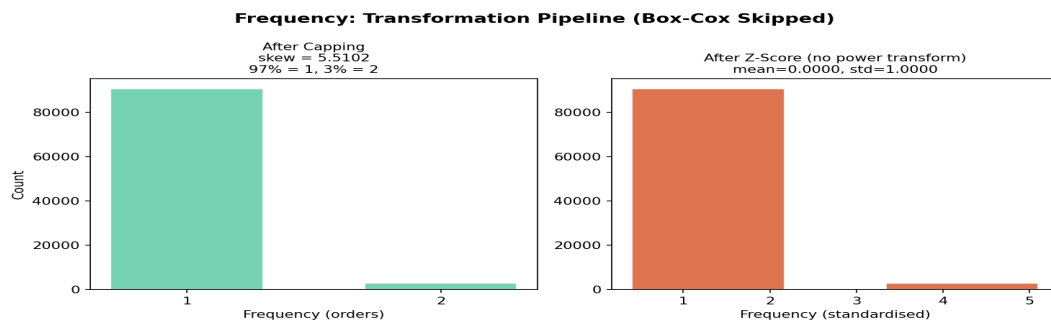


Figure 4: Frequency pipeline — no power transform applied.

Why skipped: After capping, frequency has only **2 unique values** (1 and 2). Applying Box-Cox to a near-binary feature is mathematically degenerate:

- The MLE optimiser finds an extreme λ (e.g., $\lambda = -52$) because the likelihood surface is flat — there is no meaningful 'shape' to normalise in a two-point distribution.
- The transformed skewness remains unchanged (5.51) because no monotonic transform can reshape a 97%/3% binary split into a symmetric distribution.
- The pipeline guards against this with a **MIN_UNIQUE_FOR_TRANSFORM = 10** threshold: any feature with fewer than 10 unique values after capping is passed through to StandardScaler unchanged.

Note: This is not a limitation of the pipeline — it reflects a genuine data characteristic: 97% of Olist customers are one-time buyers. Frequency acts as a near-binary indicator (bought once vs bought again), and StandardScaler handles binary features correctly by centring and scaling.

Value	Count	Percentage	After Z-score
1 (one-time)	90,556	97.0%	-0.171
2 (repeat)	2,801	3.0%	+5.68

9. Z-Score Standardisation

After power transforms, each feature has a different scale and unit. Z-score standardisation (StandardScaler) centres each feature to mean = 0 and scales to standard deviation = 1:

$$z = (x - \mu) / \sigma$$

9.1 Why Z-Score After Power Transforms?

- **K-Means uses Euclidean distance** — If features have different scales, the feature with the largest variance dominates distance calculations. A 100-unit difference in monetary would overwhelm a 1-unit difference in frequency, making K-Means cluster almost entirely on monetary.
- **Power transforms fix shape, not scale** — Box-Cox makes distributions symmetric but doesn't equalise their variances. Recency after Box-Cox ranges [7.28, 24.31] while monetary

ranges $[-1.58, -0.63]$. Without standardisation, recency would dominate by 25:1.

- **PCA requires centred data** — PCA computes the covariance matrix, which assumes features are centred at zero. Without centring, the first principal component would capture the mean offset rather than the direction of maximum variance.

9.2 Why Z-Score Over Other Scalers?

Method	When to Use	Why Not Here
Z-score (StandardScaler)	When features are approx. Gaussian after transform	USED — features are near-Gaussian after Box-Cox
Min-Max [0, 1]	When you need bounded outputs (e.g., neural nets)	Sensitive to outliers; compresses bulk of data into narrow range
Robust Scaler (IQR-based)	When outliers remain after cleaning	Outliers already capped; Z-score is more natural for Gaussian data

9.3 Per-Feature After Standardisation

Feature	Mean	Std	Min	Max	Pre-Z Shape
Recency	0.0000	1.0000	-1.93	1.99	Symmetric (Box-Cox, $\lambda=0.45$)
Frequency	0.0000	1.0000	-0.17	5.68	Binary (97%/3% split, no transform)
Monetary	0.0000	1.0000	-2.98	3.85	Symmetric (Box-Cox, $\lambda=-0.19$)

Verification: All three features now have mean ≈ 0 and std ≈ 1 , confirming equal contribution to Euclidean distance in K-Means and equal weight in PCA's covariance matrix.

10. Interaction Terms — Why Not Used

Interaction terms (e.g., frequency $\times$ monetary, recency $\times$ frequency) were considered but deliberately not included. Here is the reasoning:

10.1 K-Means Cannot Exploit Interactions

K-Means partitions data based on Euclidean distance to centroids. It draws linear decision boundaries in feature space. Adding interaction terms would increase dimensionality from 3 to 6+ features (R, F, M, R $\times$ F, R $\times$ M, F $\times$ M) without enabling K-Means to discover more complex boundaries — it would still draw hyperplanes, just in a higher-dimensional space. The extra dimensions would dilute cluster separation (curse of dimensionality) and make the results harder to interpret.

10.2 PCA Already Captures Linear Combinations

PCA decomposes the feature space into orthogonal linear combinations of R, F, and M. The principal components are weighted combinations of the original features — PC1 = 0.70 $\times$ frequency + 0.70 $\times$ monetary – 0.13 $\times$ recency already captures the frequency-monetary interaction. Adding explicit interaction terms before PCA would introduce multicollinearity (F $\times$ M is correlated with both F and M), inflating the covariance matrix without adding independent information.

10.3 Only 3 Original Features

With only 3 input features, the feature space is already low-dimensional. Interaction terms are most valuable when the feature space is rich enough to hide nonlinear relationships (e.g., 50+ features in a tabular ML model). In a 3-feature RFM space, the relationships are transparent and well-captured by PCA's linear combinations.

10.4 Interpretability Would Suffer

One of the project's goals is that segment profiles are interpretable by non-technical stakeholders. A cluster described as 'high recency, low frequency, medium monetary' is actionable. A cluster described as 'high recency $\times$ frequency interaction' is not. Adding interaction terms would obscure the SHAP explanations and make the segment heatmap unreadable.

Note: If the clustering algorithm were a kernel-based method (e.g., Spectral Clustering) or a tree-based model (e.g., using cluster labels as a supervised target), interaction terms would be handled implicitly by the algorithm. For K-Means + PCA, they add noise rather than signal.